

```
pip install catboost
```

```
Collecting catboost
  Downloading catboost-1.2.10-cp312-cp312-manylinux2014_x86_64.whl.metadata (1.2 kB)
Requirement already satisfied: graphviz in /usr/local/lib/python3.12/dist-packages (from catboost) (0.21)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (from catboost) (3.10.0)
Requirement already satisfied: numpy<3.0,>=1.16.0 in /usr/local/lib/python3.12/dist-packages (from catboost) (2.0.2)
Requirement already satisfied: pandas<4.0,>=0.24 in /usr/local/lib/python3.12/dist-packages (from catboost) (2.2.2)
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages (from catboost) (1.16.3)
Requirement already satisfied: plotly in /usr/local/lib/python3.12/dist-packages (from catboost) (5.24.1)
Requirement already satisfied: six in /usr/local/lib/python3.12/dist-packages (from catboost) (1.17.0)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas<4.0,>=0.24->catboost) (2.9.0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas<4.0,>=0.24->catboost) (2024.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas<4.0,>=0.24->catboost) (2024.2)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->catboost) (1.3.0)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib->catboost) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib->catboost) (4.53.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->catboost) (1.5.0)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib->catboost) (26.0)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib->catboost) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->catboost) (3.3.1)
Requirement already satisfied: tenacity>=6.2.0 in /usr/local/lib/python3.12/dist-packages (from plotly->catboost) (9.1.4)
Downloading catboost-1.2.10-cp312-cp312-manylinux2014_x86_64.whl (97.1 MB)
----- 97.1/97.1 MB 8.6 MB/s eta 0:00:00

Installing collected packages: catboost
Successfully installed catboost-1.2.10
```

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import pickle
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report
from sklearn.impute import SimpleImputer
from imblearn.over_sampling import SMOTE
from catboost import CatBoostClassifier

print("Starting EDA & Modeling Pipeline..")

# =====
# 1. Load Dataset & Basic Info
# =====
df = pd.read_csv('dataset.csv')

print("\nMissing Values Before Imputation:\n", df.isnull().sum())

# =====
# 2. Feature Engineering & Encoding
# =====
# Encoding target and categorical features
# Assuming 'is_patient' 1 = Liver Patient, 2 = Non-Liver Patient
df['is_patient'] = df['is_patient'].map({1: 1, 2: 0})
df['gender'] = df['gender'].map({'Male': 1, 'Female': 0})

# Create new features BEFORE splitting so they are included in the model
# Added 1e-5 to avoid division by zero errors
df['bilirubin_ratio'] = df['direct_bilirubin'] / (df['tot_bilirubin'] + 1e-5)
df['enzyme_ratio'] = df['sgpt'] / (df['sgot'] + 1e-5)
df['protein_balance'] = df['albumin'] / (df['tot_proteins'] + 1e-5)

# =====
# 3. Split Features & Target
# =====
X = df.drop(columns=['is_patient'])
y = df['is_patient']

# =====
# 4. Train-Test Split
# =====
# Stratify ensures the ratio of classes remains consistent in both splits
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# =====
# 5. Prevent Data Leakage: Impute Missing Values
# =====
# Fit ONLY on training data, then transform both train and test
imputer = SimpleImputer(strategy='mean')
X_train_imputed = pd.DataFrame(imputer.fit_transform(X_train), columns=X_train.columns)
```

```

X_test_imputed = pd.DataFrame(imputer.transform(X_test), columns=X_test.columns)

# =====
# 6. Handle Class Imbalance (SMOTE)
# =====
# Apply SMOTE ONLY to training data
smote = SMOTE(random_state=42)
X_train_res, y_train_res = smote.fit_resample(X_train_imputed, y_train)

# =====
# 7. Random Forest Model with GridSearchCV
# =====
print("\nTraining Random Forest with GridSearchCV...")
param_grid = {
    'n_estimators': [100, 200],
    'max_depth': [5, 10, None]
}

grid = GridSearchCV(RandomForestClassifier(random_state=42), param_grid, cv=3)
grid.fit(X_train_res, y_train_res)

# Extract the best model from GridSearch
best_rf_model = grid.best_estimator_
print(f"Best RF Parameters: {grid.best_params_}")

# Evaluate Random Forest
y_pred_rf = best_rf_model.predict(X_test_imputed)
print("\nRandom Forest Accuracy:", accuracy_score(y_test, y_pred_rf))
print("\nRandom Forest Classification Report:\n", classification_report(y_test, y_pred_rf))

# Feature Importance
feature_importance = best_rf_model.feature_importances_
importance_df = pd.DataFrame({
    'Feature': X.columns,
    'Importance': feature_importance
}).sort_values(by='Importance', ascending=False)
print("\nFeature Importances:\n", importance_df.head())

# =====
# 8. CatBoost Model
# =====
print("\nStarting CatBoost Model Training...")
# Removed class_weights={0:1, 1:1} as SMOTE already balances the training data
cat_model = CatBoostClassifier(random_state=42, verbose=0)
cat_model.fit(X_train_res, y_train_res)

# Evaluate CatBoost
y_pred_cat = cat_model.predict(X_test_imputed)
print("\nCatBoost Accuracy:", accuracy_score(y_test, y_pred_cat))
print("\nCatBoost Classification Report:\n", classification_report(y_test, y_pred_cat))

# =====
# 9. SAVE MODEL & PREPROCESSORS FOR DEPLOYMENT
# =====
# Save the models, the imputer, and the expected feature order
deployment_assets = {
    'rf_model': best_rf_model,
    'catboost_model': cat_model,
    'imputer': imputer,
    'feature_order': X.columns.tolist()
}

with open("model.pkl", "wb") as f:
    pickle.dump(deployment_assets, f)

print("\n🟢 model.pkl saved successfully with models, imputer, and feature order.")

```

Starting EDA & Modeling Pipeline...

Missing Values Before Imputation:

age	0
gender	0
tot_bilirubin	0
direct_bilirubin	0
tot_proteins	0
albumin	0
ag_ratio	0
sgpt	0
sgot	0
alkphos	4
is_patient	0
dtype: int64	

Training Random Forest with GridSearchCV...

Best RF Parameters: {'max_depth': None, 'n_estimators': 200}

Random Forest Accuracy: 0.717948717948718

Random Forest Classification Report:

	precision	recall	f1-score	support
0	0.52	0.44	0.48	34
1	0.78	0.83	0.81	83
accuracy			0.72	117
macro avg	0.65	0.64	0.64	117
weighted avg	0.71	0.72	0.71	117

Feature Importances:

	Feature	Importance
4	tot_proteins	0.116474
6	ag_ratio	0.111898
5	albumin	0.093672
2	tot_bilirubin	0.085362
0	age	0.083076

Starting CatBoost Model Training...

CatBoost Accuracy: 0.717948717948718

CatBoost Classification Report:

	precision	recall	f1-score	support
0	0.52	0.44	0.48	34
1	0.78	0.83	0.81	83
accuracy			0.72	117
macro avg	0.65	0.64	0.64	117
weighted avg	0.71	0.72	0.71	117

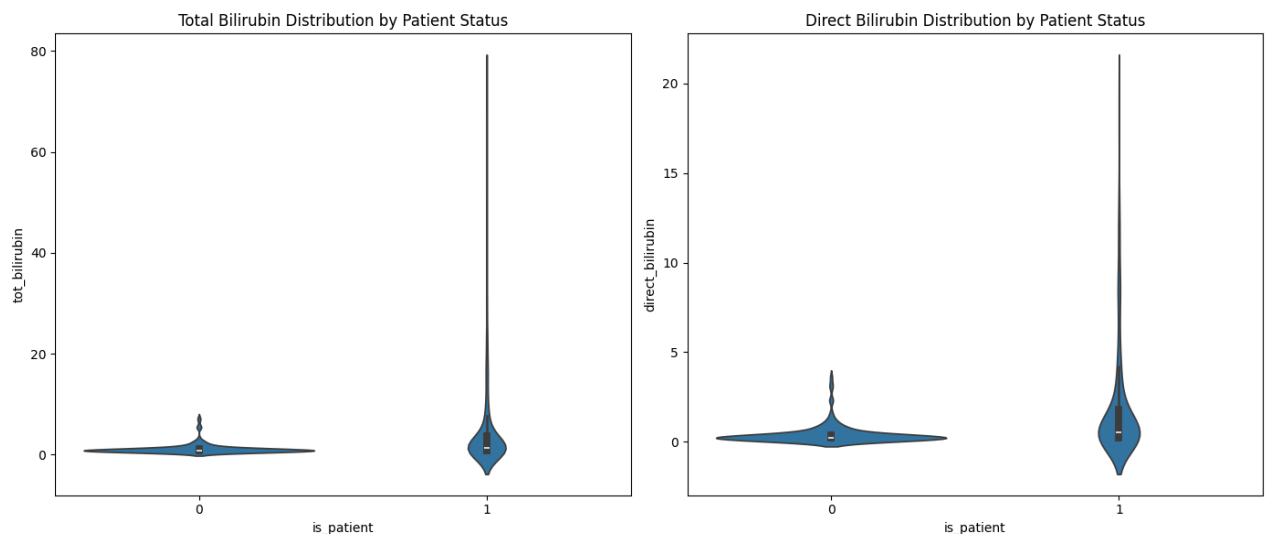
✅ model.pkl saved successfully with models, imputer, and feature order.

```
print("\n--- Additional Plots ---\n")
```

```
# 1. Violin plots for Bilirubin levels by patient status
plt.figure(figsize=(14, 6))
plt.subplot(1, 2, 1)
sns.violinplot(x='is_patient', y='tot_bilirubin', data=df)
plt.title('Total Bilirubin Distribution by Patient Status')

plt.subplot(1, 2, 2)
sns.violinplot(x='is_patient', y='direct_bilirubin', data=df)
plt.title('Direct Bilirubin Distribution by Patient Status')
plt.tight_layout()
plt.show()
```

--- Additional Plots ---



```
# 2. Scatter plot for SGPT vs SGOT, colored by patient status
plt.figure(figsize=(10, 6))
```

```
sns.scatterplot(x='sgpt', y='sgot', hue='is_patient', data=df, alpha=0.7)
plt.title('SGPT vs SGOT by Patient Status')
plt.xlabel('SGPT (Alanine Aminotransferase)')
plt.ylabel('SGOT (Aspartate Aminotransferase)')
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```

